

PawWatch

App game ato simulation social selamatan wild cats.



"Rescue. Report. Earn."

The core idea: real people, real streets, real kittens — but with the dopamine loop of a game.

How it works

Report — spot a stray or abandoned kitten, drop a pin, add a photo and quick description. Takes 30 seconds.

Respond — nearby rescuers see the alert and tap "I'm on my way." The first to confirm gets bonus XP.

Rescue — mark it resolved, log where the kitten ended up (kept, vet, shelter, rehomed). Community sees the outcome.

Earn — XP, badges, and a leaderboard per city/zone. Levels have real titles: *Kitten Spotter* → *Field Scout* → *Rescue Ranger* → *Guardian*.

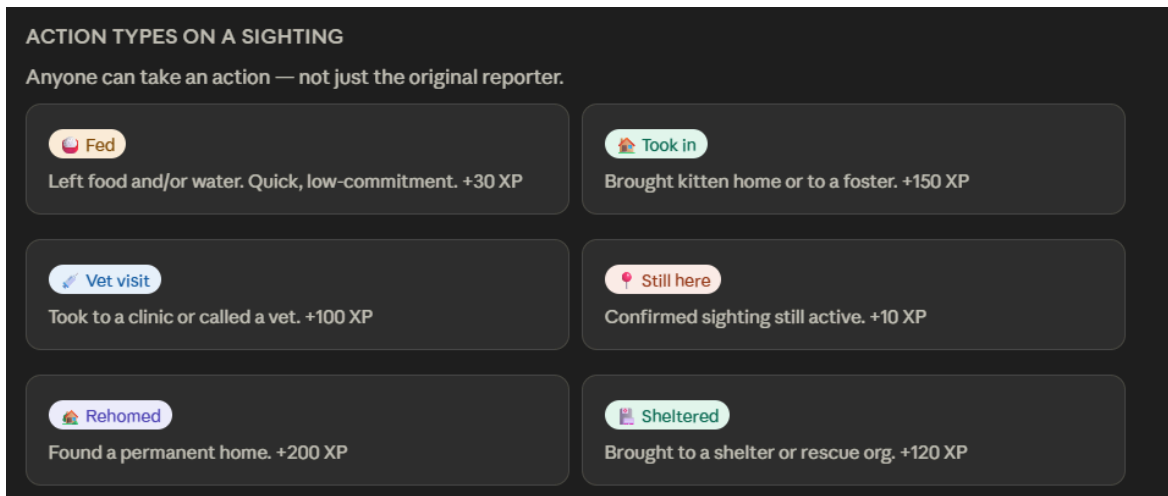
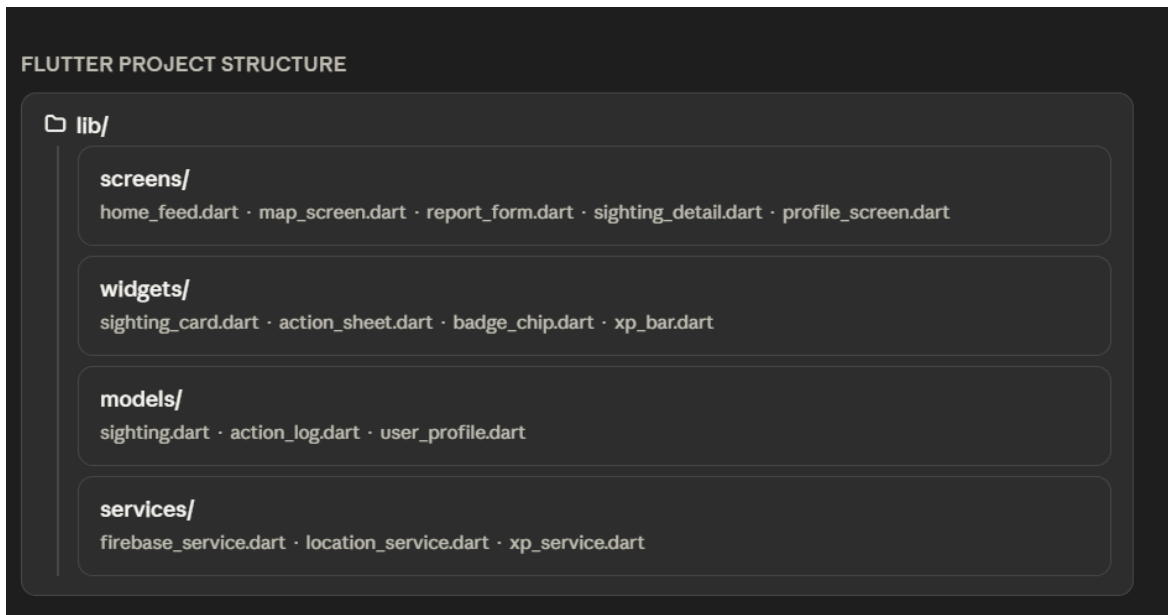
Why this works in Indonesia(Problem Statement)

Jakarta and other Indonesian cities have massive stray cat populations and a huge culture of caring about them — but zero coordination tools. PawWatch fills that gap while making it social and rewarding.

What you'd build in Flutter

- Map screen (Google Maps Flutter plugin)
- Post/feed screen (like a community board)
- Report form (photo + pin + description)
- Profile + XP/badge system

- Basic backend (Firebase is perfect for this — free tier handles early users easily)



"okay but whats the argument if its only for cats mainly?"

That's a fair concern to think about early. Here are both sides:

The argument against (the worry)

- Feels niche — "what about dogs? rabbits? birds?"

- Investors or clients might see it as limited scope
 - Could alienate users who care about other animals
-

The argument for staying cats-only (and it's strong)

Focus is a feature, not a bug. The most successful apps won — in the early days — by being ruthlessly specific. Instagram was just photos. Twitter was just short text. Gojek started as just motorbike taxis in Jakarta.

Specific arguments for cats specifically:

- Cats are by far the **most common** urban stray animal globally — and especially in Indonesia, Southeast Asia, Middle East, Japan
 - Cat welfare communities online are **massive and passionate** — they share, they organize, they donate
 - "Stray cat" is a universal, immediately understood problem. Everyone has seen one. Everyone has felt something about one.
 - A cat-specific app can have **cat-specific features** — feral colony tracking, TNR (trap-neuter-return) coordination, kitten season alerts — that a generic "all animals" app would water down
 - Cat people are **loyal users**. They don't just use apps, they evangelize them
-

The smart answer if anyone asks

"We started with cats because it's the highest-volume urban stray problem and has the strongest existing community. The architecture supports expanding to other animals — it's just a category tag in the data model."

That's honest, strategic, and shows you've thought it through.

Cats-only is a strength. Own it.

User Story

Epic 1 — Authentication

PW-001

As a new user, I want to sign up with my email and password so that I can create a PawWatch account and start reporting sightings.

- ✓ User can register with email + password
- ✓ Display name is set on first login
- ✓ User profile is created in Firestore on signup
- ✓ Validation shows error on duplicate email

Must have SP: 3

PW-002

As a returning user, I want to log in with my credentials so that I can access my profile and past activity.

- ✓ User can log in with email + password
- ✓ Wrong credentials shows clear error
- ✓ Session persists after app restart

Must have SP: 2

Epic 2 — Sighting reporting

PW-003

As a user, I want to report a stray kitten sighting with a photo, description, and my current location so that other rescuers nearby can find it.

- ✓ Form captures photo, description, and GPS location
- ✓ Location auto-fills from device GPS
- ✓ User can manually adjust pin on map
- ✓ Sighting saved to Firestore with status "open"
- ✓ Reporter earns +20 XP on submission

Must have SP: 5

PW-004

As a user, I want to mark a sighting as urgent so that rescuers know the kitten needs immediate help.

- ✓ Urgency toggle available in report form
- ✓ Urgent sightings show a distinct badge on feed and map
- ✓ Urgent sightings appear at top of nearby feed

Must have SP: 2

● Epic 3 — Rescue actions

PW-005

As a user, I want to log that I fed a stray kitten so that the community knows it has been cared for even if no one can take it in.

- ✓ "Fed" action available on any open sighting
- ✓ Optional note and photo can be added
- ✓ Sighting status updates to "fed"
- ✓ User earns +30 XP
- ✓ Action appears in sighting's activity log

Must have SP: 3

PW-006

As a user, I want to log a full rescue action (took in, vet visit, sheltered, rehomed) so that the community knows the kitten's outcome.

- ✓ Action type selector shows all 5 rescue types
- ✓ Each type awards the correct XP amount
- ✓ Sighting status updates accordingly
- ✓ Resolved sightings are visually distinct on map

Must have SP: 5

PW-007

As a user passing by a known sighting, I want to mark it as "still here" so the community knows it is still active and hasn't been resolved.

- ✓ "Still here" action available on open sightings
- ✓ Updates lastActionAt timestamp
- ✓ User earns +10 XP
- ✓ Sighting moves up in recency sort

Should have SP: 2

Epic 4 — Map & feed

PW-008

As a user, I want to see nearby sightings on a live map so that I can quickly identify where kittens need help around me.

- ✓ Map shows pins color-coded by status
- ✓ Tapping a pin opens sighting detail
- ✓ Map centers on user's current location
- ✓ Real-time updates via Firestore listener

Must have SP: 5

PW-009

As a user, I want to browse a community feed of recent sightings so that I can stay up to date without opening the map.

- ✓ Feed shows sightings sorted by recency
- ✓ Each card shows photo, status badge, distance, and time
- ✓ Pull-to-refresh works
- ✓ Tapping a card opens full sighting detail

Must have SP: 3

Epic 5 — Gamification

PW-010

As a user, I want to earn XP for every action I take so that I feel rewarded for contributing to the community.

- ✓ XP awarded matches defined amounts per action type
- ✓ XP total updates in real time on profile
- ✓ XP bar animates on earn

Must have SP: 3

PW-011

As a user, I want to level up as I earn XP so that I can see my progress and feel a sense of growth.

- ✓ Level thresholds defined (e.g. 0, 200, 500, 1000 XP...)
- ✓ Level title updates with each tier
- ✓ Level-up triggers a celebration UI moment

Must have SP: 3

PW-012

As a user, I want to earn badges for milestones so that my achievements are visible on my profile.

✓ Badges awarded automatically on trigger (first rescue, 10 feeds, etc.)

✓ Locked badges show on profile as grayed out

✓ Badge earn triggers an in-app notification

Should have

SP: 4

PW-013

As a user, I want to see a leaderboard for my city so that I can compare my contributions with other local rescuers.

✓ Leaderboard shows top 10 users by XP in same city

✓ Current user's rank always visible even if outside top 10

✓ Refreshes weekly

Could have

SP: 4

●

Epic 6 — Profile

PW-014

As a user, I want to view my profile so that I can see my rescue stats, level, and earned badges in one place.

✓ Shows display name, level, XP bar

✓ Shows total rescues, total feeds, total reports

✓ Shows earned and locked badges

Must have

SP: 3

Total backlog

14 stories · 47 story points · 9 must-have · 3 should-have · 2 could-have

SPRINT

Sprint 1 (foundation) — knock out PW-001, PW-002, PW-003, PW-008. That's auth + report + map. You have a usable app at the end of this sprint.

Sprint 2 (core loop) — PW-005, PW-006, PW-009, PW-010, PW-014. Actions, feed, XP, and profile. This is when it starts feeling like a real product.

Sprint 3 (polish + gamification) — PW-004, PW-007, PW-011, PW-012, PW-013. Urgency, levels, badges, leaderboard.

[pawwatch_full_scrum_plan.html](#)

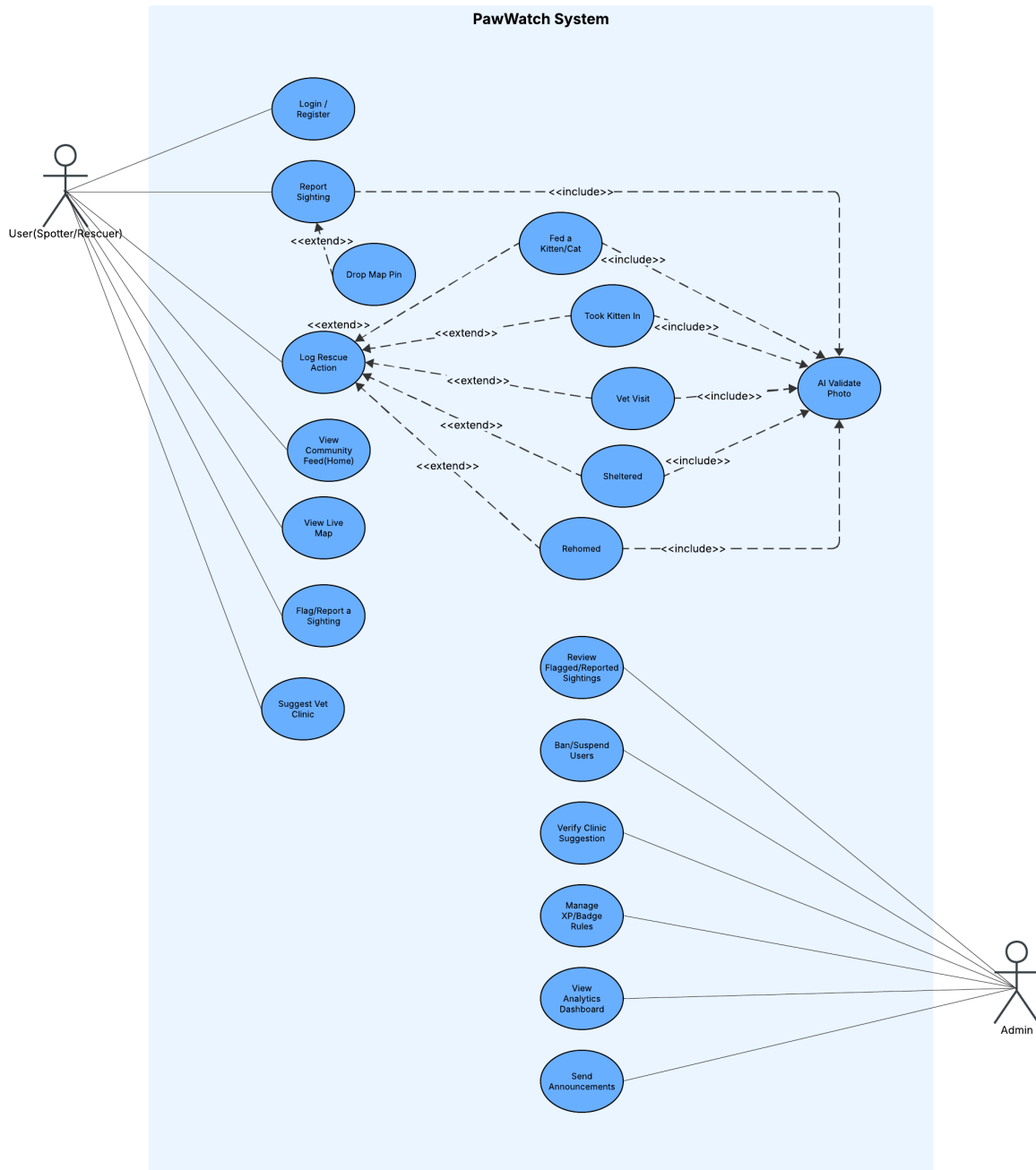
Btw klo bs pas coding implementasi prinsip oop dan struktur code modular

Use Case

okay one questin, do the admin manually verify vet clinics or could the user tell and verify the nearesst clinic too? and will that clinic have their own account on being a vet clinics app or not needed?

Option B — User can suggest, Admin approves

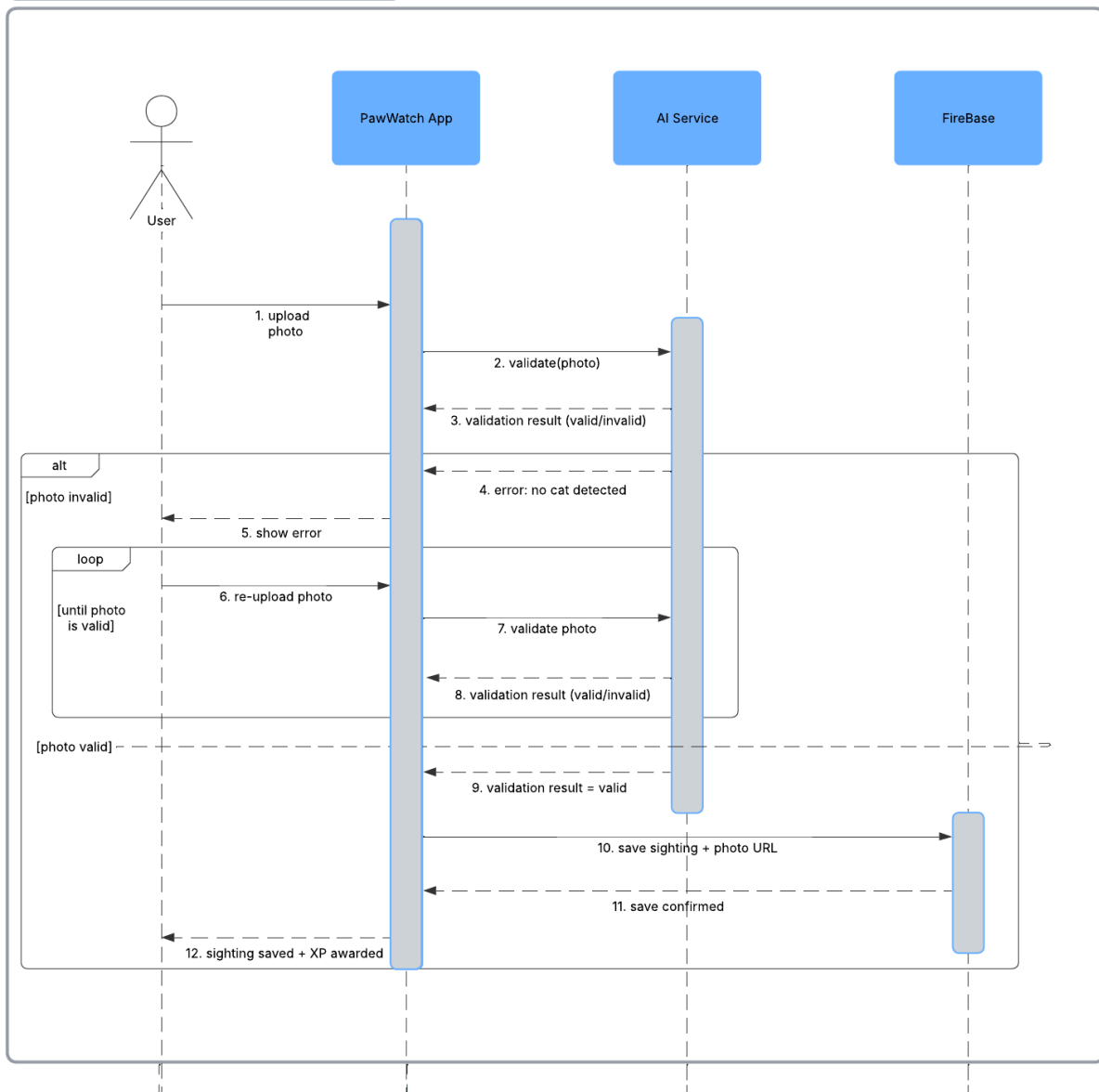
- Any user can submit a nearby clinic
- Admin just clicks approve/reject
- Crowdsourced but still moderated
- **No clinic account needed**
- More scalable, less admin work



Sequence Diagram

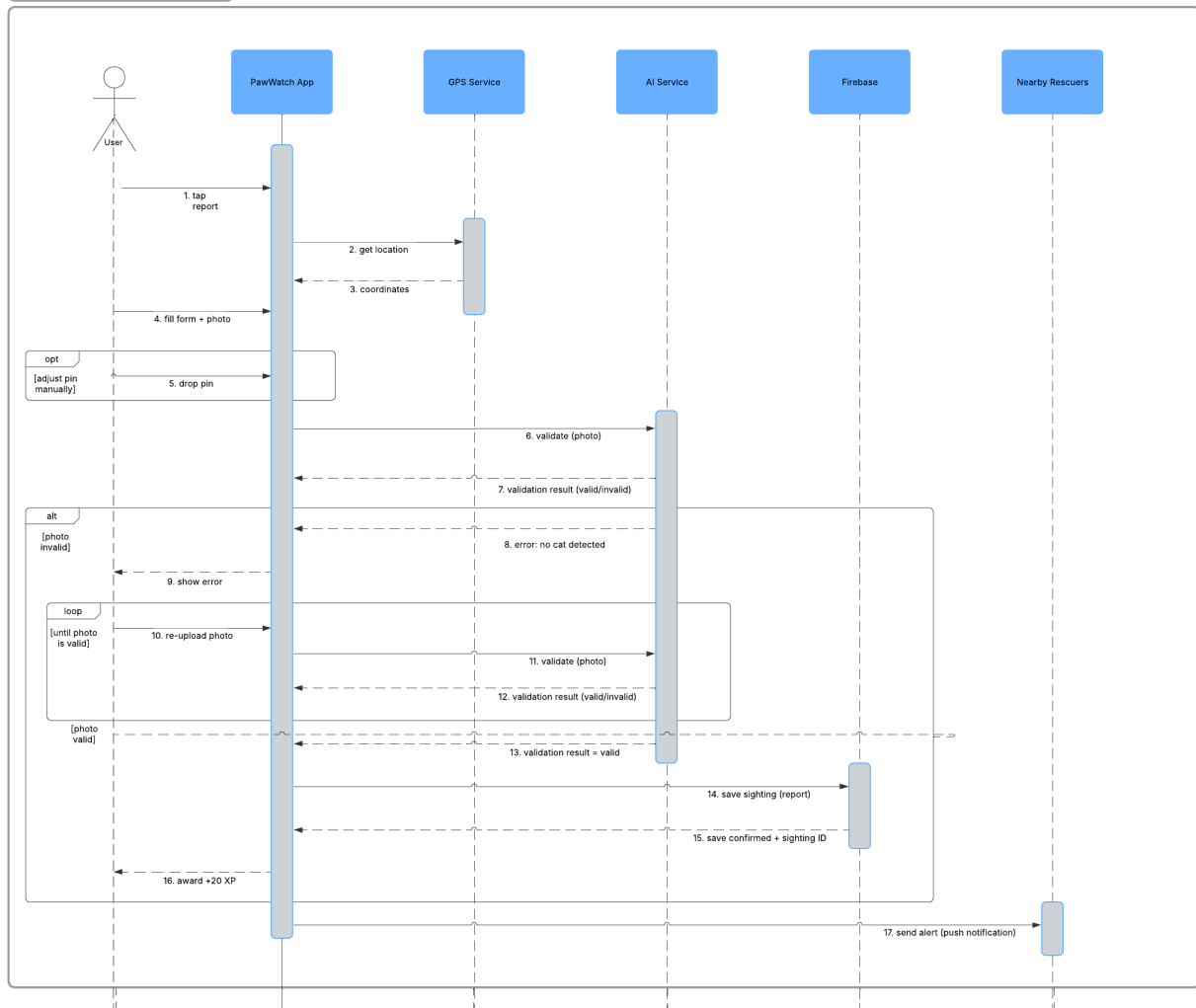
1. AI Photo Validation — User → App → AI Service → Firebase. Multiple systems, async calls, failure handling. This is the trickiest one.

AI Photo Validation



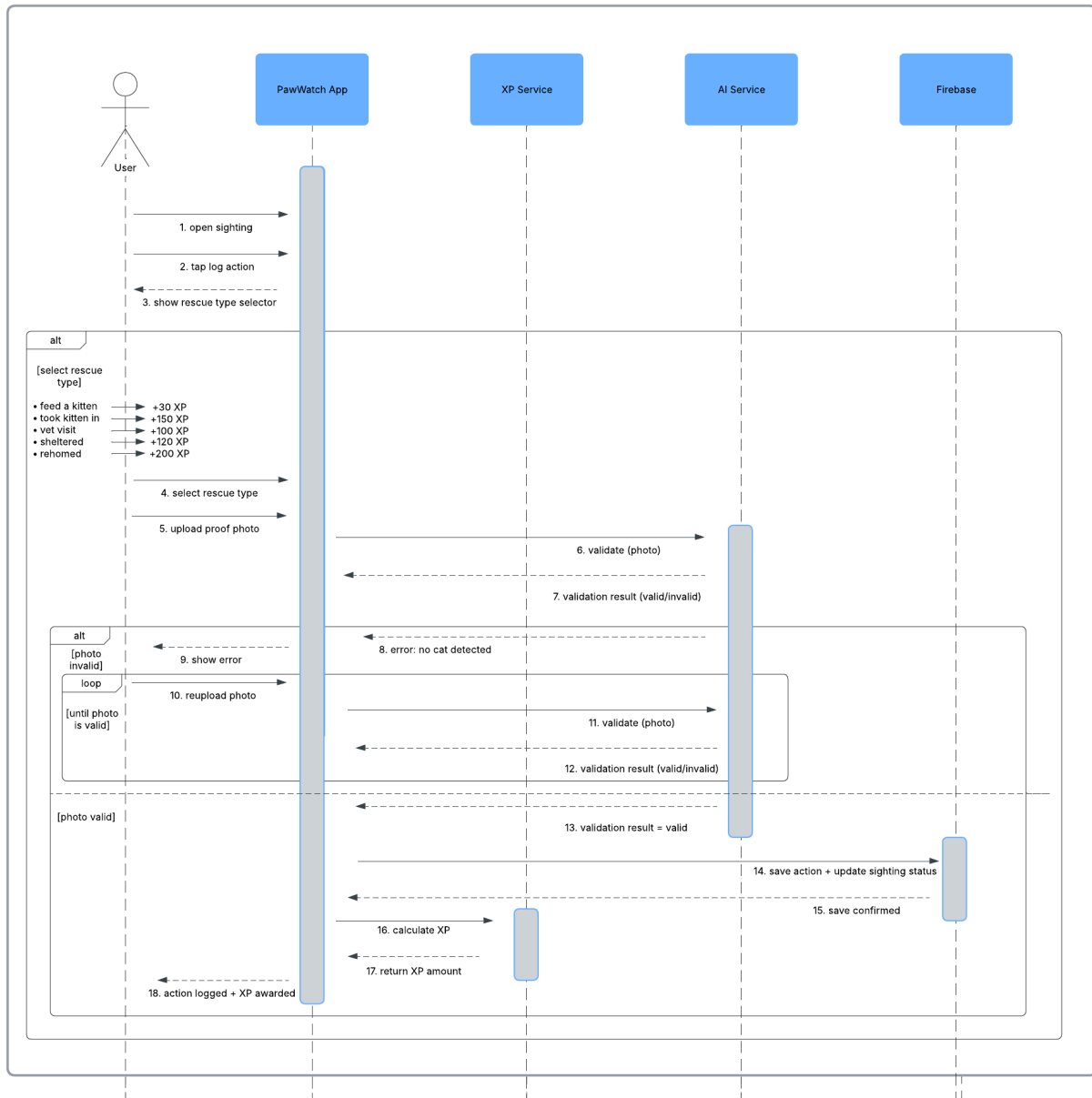
2. Report Sighting (full flow) — User → App → GPS Service → Firebase → push notification to nearby rescuers. Worth documenting because it involves 4 systems.

Report Sighting

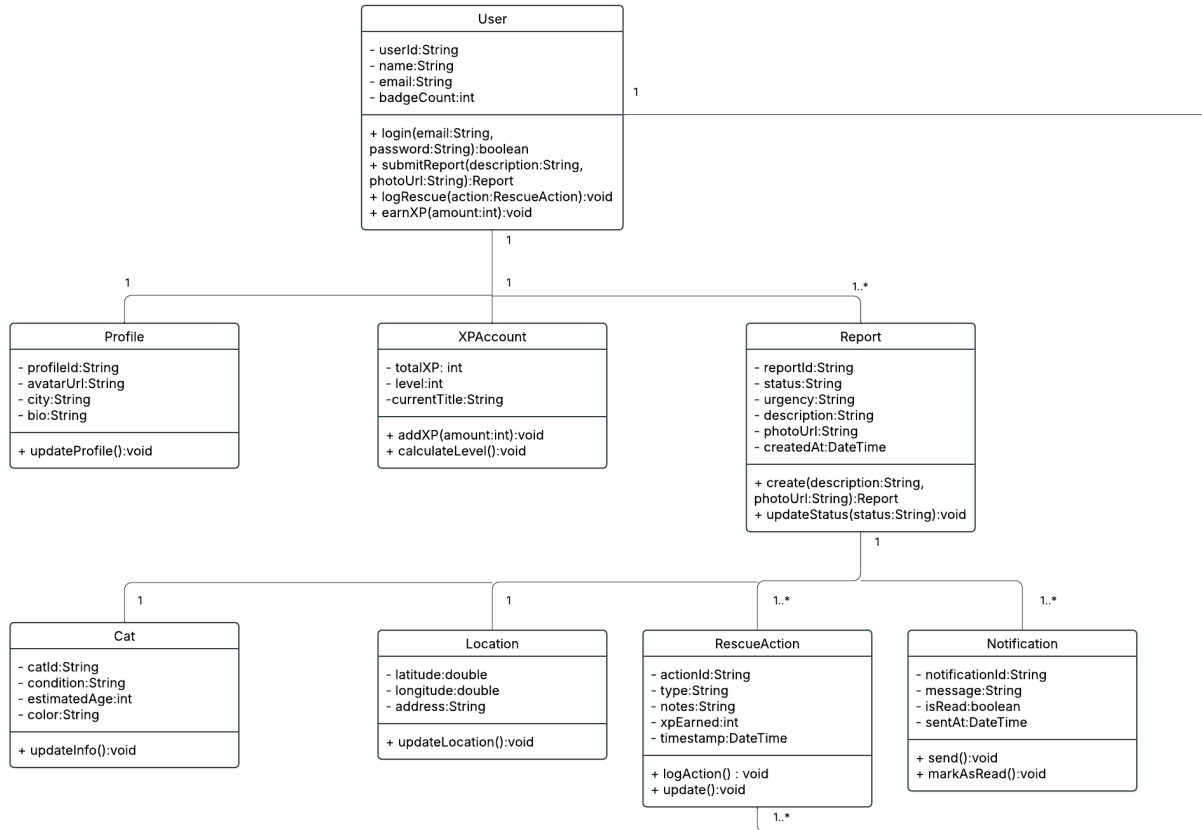


3. Log Rescue Action — User → App → AI Service → Firebase → XP Service.
 Similar complexity to photo validation but with the XP calculation added.

Log Rescue Action



Class Diagram



Explanation:

a)nums:

No worries! This is one of the most important concepts in UML. The difference is simply **how many objects can participate in the relationship**.

1 = Exactly one

There must be **one and only one** related object.

For example:

User 1 ————— 1 Profile

Meaning:

- One User has exactly one Profile.
- One Profile belongs to exactly one User.

So if User A exists:



User A **cannot** have two profiles.

1..* = One or more

This means there must be **at least one**, but there can be many.

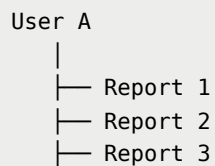
For example:



Meaning:

- One User can have one report.
- One User can have two reports.
- One User can have ten reports.

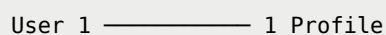
Example:



All of those reports belong to the same user.

Applying it to your PawWatch diagram

User — Profile



A user only has one profile.

User — XPAccount

User 1 — 1 XPAccount

A user only has one XP account.

User — Report

User 1 — 1..* Report

A user can submit many reports over time.

Example:

```
John
├─ Report: Cat at Mall
├─ Report: Injured Dog
└─ Report: Lost Kitten
```

Report — Notification

Report 1 — 1..* Notification

One report can generate multiple notifications.

For example:

```
Report #15
├─ Notification: Report received
├─ Notification: Volunteer assigned
└─ Notification: Rescue completed
```

Report — RescueAction

Report 1 ————— 1..* RescueAction

A single report may involve multiple rescue actions.

For example:

```
Report
├─ Volunteer A rescued cat
├─ Volunteer B transported cat
└─ Volunteer C updated medical status
```

Quick reference

Multiplicity	Meaning	Example
1	Exactly one	One user has one profile
0..1	Zero or one	A report may or may not have a location
1..*	One or more	One user has many reports
* (or 0..*)	Zero or more	A user may have no reports yet, or many

One thing to think about

For a real PawWatch app, **User → Report** is often modeled as **0..*** rather than **1..***, because a brand-new user can exist before they've submitted their first report.

Similarly:

- **User → Report** : **0..***
- **User → RescueAction** : **0..***

However, if your lecturer or your project assumes every user in the system is an active participant who has already reported or rescued at least once, then **1..*** is also a reasonable modeling choice. Many student projects simplify it this way.

b)location

Great question. In your PawWatch app, **latitude** and **longitude** are simply the GPS coordinates that tell the app **where the cat was found**.

For example, if someone reports a stray cat, the app needs to know the exact location.

Latitude

Latitude measures **how far north or south** a location is from the Equator.

- Range: **90° to +90°**
- Example:

```
-6.2088
```

This is around **Jakarta, Indonesia**.

Longitude

Longitude measures **how far east or west** a location is from the Prime Meridian (Greenwich, England).

- Range: **180° to +180°**
- Example:

```
106.8456
```

This is also around Jakarta.

Together

A location is represented by **both** values.

For example:

```
Location
-----
latitude  = -6.2088
longitude = 106.8456
address   = "Central Jakarta"
```

Your app can then display a pin on a map exactly at that position.

📍 Like Google Maps:



●
Latitude: -6.2088
Longitude: 106.8456

Why does PawWatch need them?

When a user submits a report, the app can:

- 📍 Save the exact location where the cat was seen.
- 🗺️ Show the report on a map.
- 🚑 Find nearby rescuers.
- 🔔 Send notifications to volunteers near that location.

That's why your `Location` class has:

```
Location
-----
latitude : double
longitude : double
address : String
```

- `latitude` → precise north/south coordinate.
- `longitude` → precise east/west coordinate.
- `address` → human-readable location (e.g., "Jl. Sudirman No. 15, Jakarta").

Why is the type `double` ?

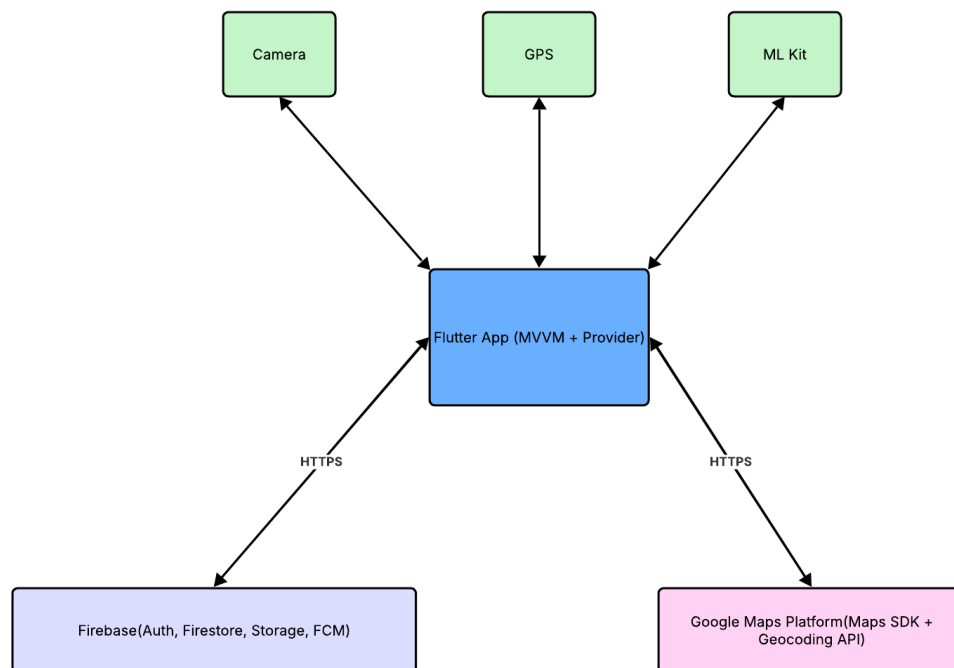
Coordinates contain decimal values, for example:

```
latitude = -6.208763
longitude = 106.845599
```

A `double` can store these decimal numbers accurately, whereas an `int` cannot.

So your `Location` class is correctly modeled for a location-based app like PawWatch.

System Architecture



Component	Simple Explanation
Flutter App (MVVM + Provider)	The main mobile application that users interact with. It is built using Flutter and follows the MVVM (Model view-view model) architecture with Provider for state management.
Camera	Allows users to capture or upload photos of stray cats when creating a report.
GPS	Retrieves the user's current location so the app can record where the cat was found and display it on the map.

Component	Simple Explanation
ML Kit	A machine learning tool used to validate whether the uploaded image contains a cat before the report is submitted.
Firebase	The cloud backend that stores and manages the application's data and services.
Auth	Firebase Authentication. Handles user registration, login, and account verification.
Firestore	A cloud database that stores application data such as user profiles, reports, rescue logs, and XP.
Storage	Cloud storage used to save uploaded cat photos.
FCM	Firebase Cloud Messaging. Sends push notifications to nearby rescuers or users.
Google Maps Platform	Provides map-related services used in the application.
Maps SDK	Displays interactive Google Maps inside the Flutter app.
Geocoding API	Converts GPS coordinates into readable addresses (or vice versa).
HTTPS <i>(arrow label)</i>	A secure internet communication protocol used to exchange data between the Flutter app and cloud services (Firebase and Google Maps Platform).

User-facing pages

Auth

1. **Login** — email/password sign in
2. **Register/Sign Up** — email/password, sets display name

Core

3. **Home Feed** — community board, sightings sorted by recency, pull-to-refresh
4. **Map Screen** — live map with color-coded pins by status, Google Maps SDK
5. **Report Form** — photo + pin + description + urgency toggle, GPS auto-fill
6. **Sighting Detail** — full view of a single sighting: photo, status, action history, "I'm on my way," action buttons (Fed, Vet visit, Took in, Rehomed, Sheltered, Still here)

Profile & Gamification

7. **Profile Screen** — display name, level, XP bar, stats (rescues/feeds/reports), badges

8. **Badges Screen** (could be a section of Profile, or standalone) — earned + locked badges

9. **Leaderboard** — top 10 per city/zone, current user's rank

Supporting

10. **Suggest Vet Clinic** — form for users to submit a nearby clinic (Option B: crowdsourced, admin-approved)

11. **Notifications** — list of alerts (nearby sighting, someone responded, badge earned, etc.)

Admin-facing pages (from your use case diagram)

1. **Admin Dashboard** — analytics overview
2. **Review Flagged/Reported Sightings**
3. **Manage Users** — ban/suspend
4. **Verify Clinic Suggestions** — approve/reject queue
5. **Manage XP/Badge Rules**
6. **Send Announcements**

That's **11 user-facing screens** and **6 admin screens** — matches well with your existing `screens/` folder (home_feed, map_screen, report_form, sighting_detail, profile_screen) plus a few you hadn't scaffolded yet (auth, leaderboard, clinic suggestion, notifications, admin panel).

A couple of things worth deciding before you wireframe:

- Is **admin** a separate app/web dashboard, or screens inside the same Flutter app gated by role? Most teams do web/admin separately since it's low-traffic and easier to build in something like a simple React admin panel or even Firebase console + Cloud Functions.

- Does **Notifications** need its own full page, or just a dropdown/panel? Given FCM push is already in your architecture, a simple in-app notification list page makes sense.

Want me to sketch the navigation structure next (bottom nav vs drawer, which screens are modals vs full pages, tab structure)?